Vorgehensmodell und Entwicklungsplan

- CrossNote

Datum:	14.01.2026
Version:	1.2
Prüfer:	Kai Seitz
Freigegeben von:	Jakob Scheuermeyer (Qualitätsmanager)

Changelog:

1.0: grobe Ausarbeitung
1.1: erste fertige Ausarbeitung
1.2: Deckblatt

Gerhard Uebe, Nicolas Kauer, Jakob Scheuermeyer, Kai Seitz

Vorgehensmodell: Scrum

Rahmen:

- Zeitraum: 6 Monate → ca. 12 Sprints (2 Wochen).
- Artefakte: Product Backlog, Sprint Backlog, Definition of Ready/Done, Burndown, Review-Protokolle.
- Tools (Beispiel): GitHub Projects, Issues, Actions (CI), SonarLint, Playwright/Cypress.

Phasen je Sprint (wiederholt):

1. Backlog-Pflege (Rolling Refinement)
2. Sprint Planning
3. Daily (max. 15 min)
4. Implementierung + Code Review + Tests + CI/CD
5. Sprint Review (Abnahme)
6. Sprint Retrospektive (Verbesserungen)

Einmalig am Anfang (Sprint 0 / Kick-off):

- Projektsetup, Architektur-Skizze, Systemgrenzen (Heimnetz, geringe Serverleistung, beschränkter Datei), Risiko-/Release-Plan, MVP-Scope finalisieren.

Definition of Done (DoD, Beispiele):

- Build grün (CI), Unit-/E2E-Tests grün, Code-Review erfolgt, Sicherheits-Checks ohne kritische Findings, Doku aktualisiert
- Definition von kritischen Findings:
    - Unbeschränkter Datei-Upload (fehlende Content-Prüfung/Größenlimit, ausführbare Dateien) → RCE/DoS-Risiko → Blocker

    - Datenverlust beim Sync (Änderungen gehen reproduzierbar verloren) → Blocker
    - Korruption von Notizen/Anhängen (inkonsistente Versionen, defekte Binärdaten) → Blocker
    - Kein lauffähiger Restore (Backup vorhanden, aber Wiederherstellung scheitert) → Blocker
    - CI/CD rot (Build/Test fehlgeschlagen) → Blocker
    - Testabdeckung Kernmodule < 80 % oder fehlende E2E-Tests für kritische Flows (Login, CRUD, Sync) → Blocker

Akzeptanzkriterien (MVP-Beispiele):

- Notiz-CRUD funktioniert online & offline (späterer Sync).
- Sync zwischen Geräten < 5 s nach Verbindung.
- Suche < 300 ms bei 1.000 Notizen (clientseitig).
- Läuft auf Windows, Linux, Android; Server ressourcenschonend (≤ 1 vCPU, ≤ 512 MB RAM Demo).

Rollen-Mapping (RACI-Kurz):

- PO (Gerhard): Bedarfsermittlung, Priorisierung, Release Termine, Feedback berücksichtigen, Stakeholder, Management, Abnahme
- Scrum-Master (Nicolas): Teamunterstützung, Prozessunterstützung, Problemmanagement, Kontinuierliche Verbesserung, Scrum Trainer
- Dev (alle): Arbeitsaufgaben definieren, Qualität sicherstellen, Aufwände schätzen, Produkt interdisziplinär entwickeln
- QM (Jakob): Qualität, Metriken, Reviews

Tools und Programmiersprachen:

- Kotlin für die Android Umgebung
- Java für alle App Anwendungen
- Python für den Server
- VS-Code als IDE
- Git als Versionsverwaltung
- Github als Code-Austausch unter den Developern

Design:

Farben (Hex-Werte):

- Blau: 307EC4

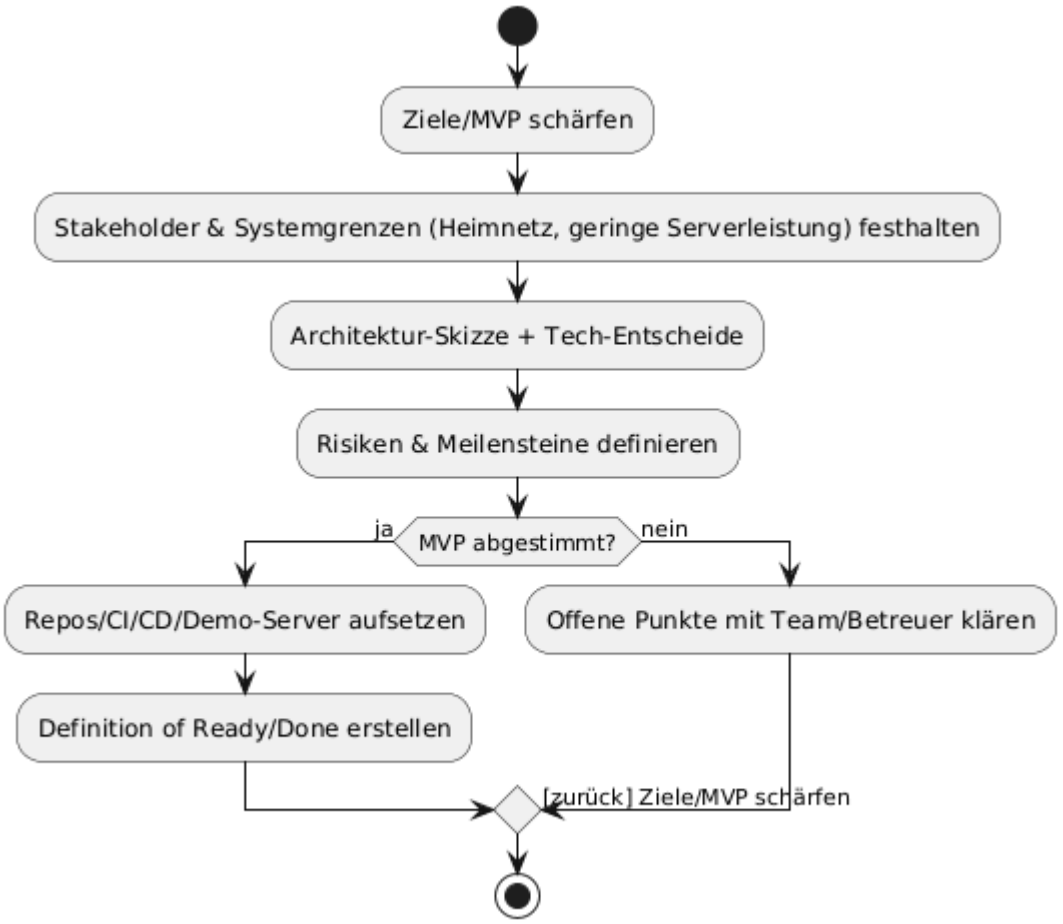
Layout: inspiriert von Clevnote

Metriken:

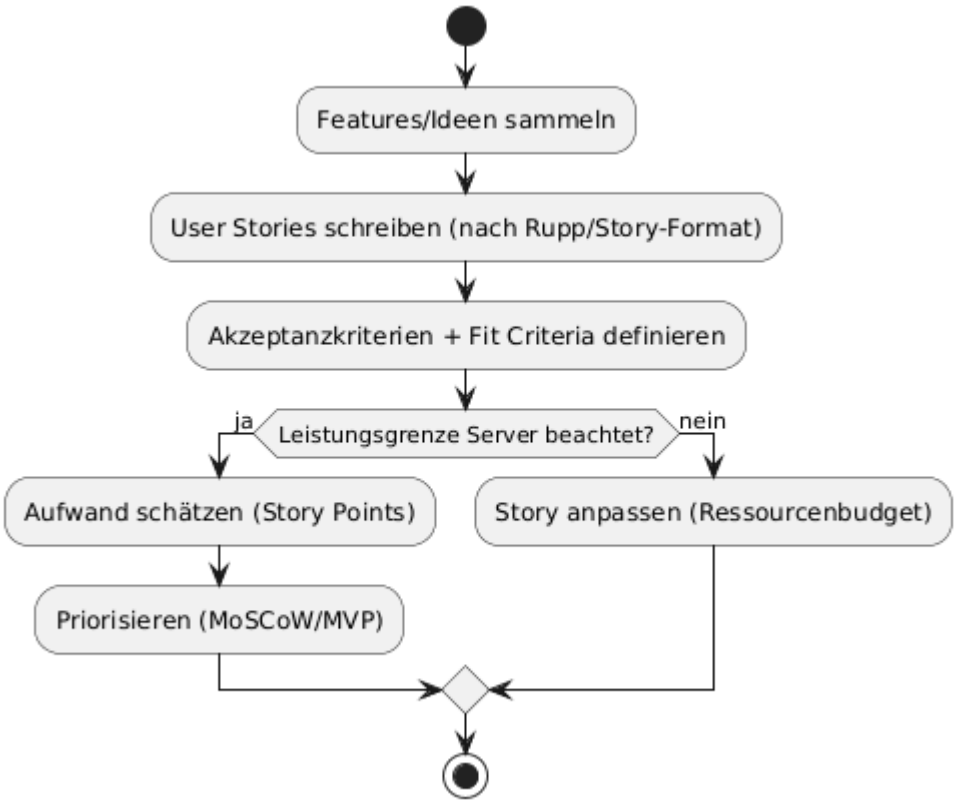
- Testabdeckung (Coverage): Anteil getesteter Codezeilen (Unit + Integration Tests) -> ≥ 80 %
- Datenverlustquote: Anteil verlorener oder beschädigter Notizen -> ~ 0 %
- App-Startzeit: Zeit vom Start bis zur Nutzbarkeit der App -> < 2 Sekunden
- Code Dokumentationsgrad: Anteil dokumentierter Module / Klassen -> ≥ 90 %
- Plattformabdeckung: getestete Plattformen (Windows, Linux, Android) -> alle 3 funktionieren fehlerfrei

Aktivitätsdiagramme:

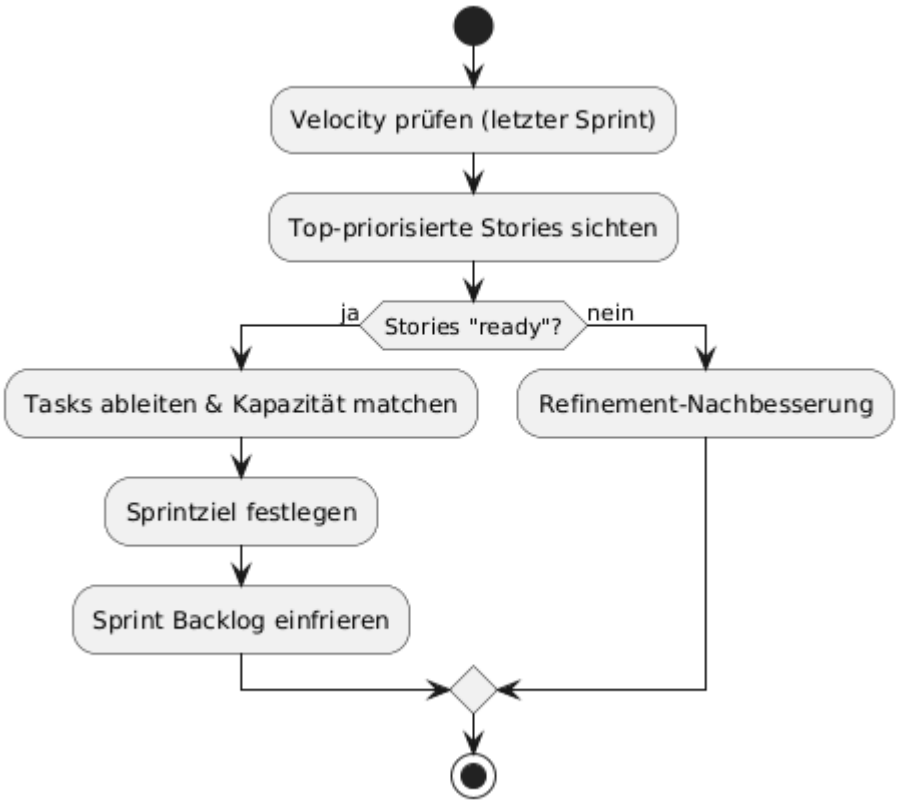
1) Kick-Off und Projektsetup



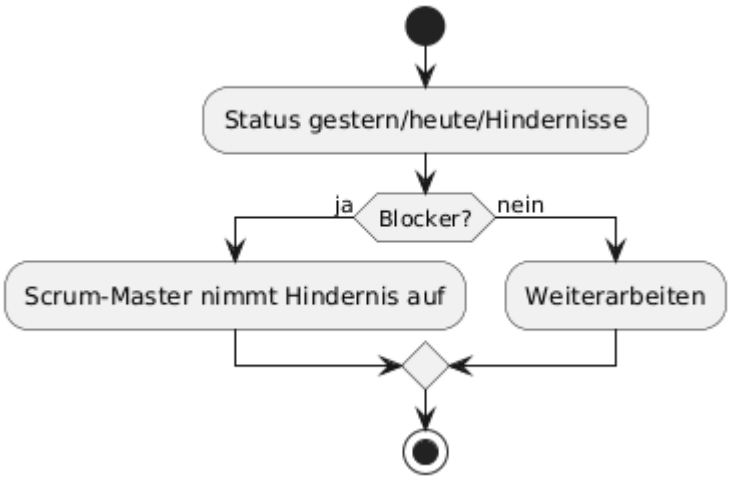
2) Backlog-Pflege / Refinement (laufend)



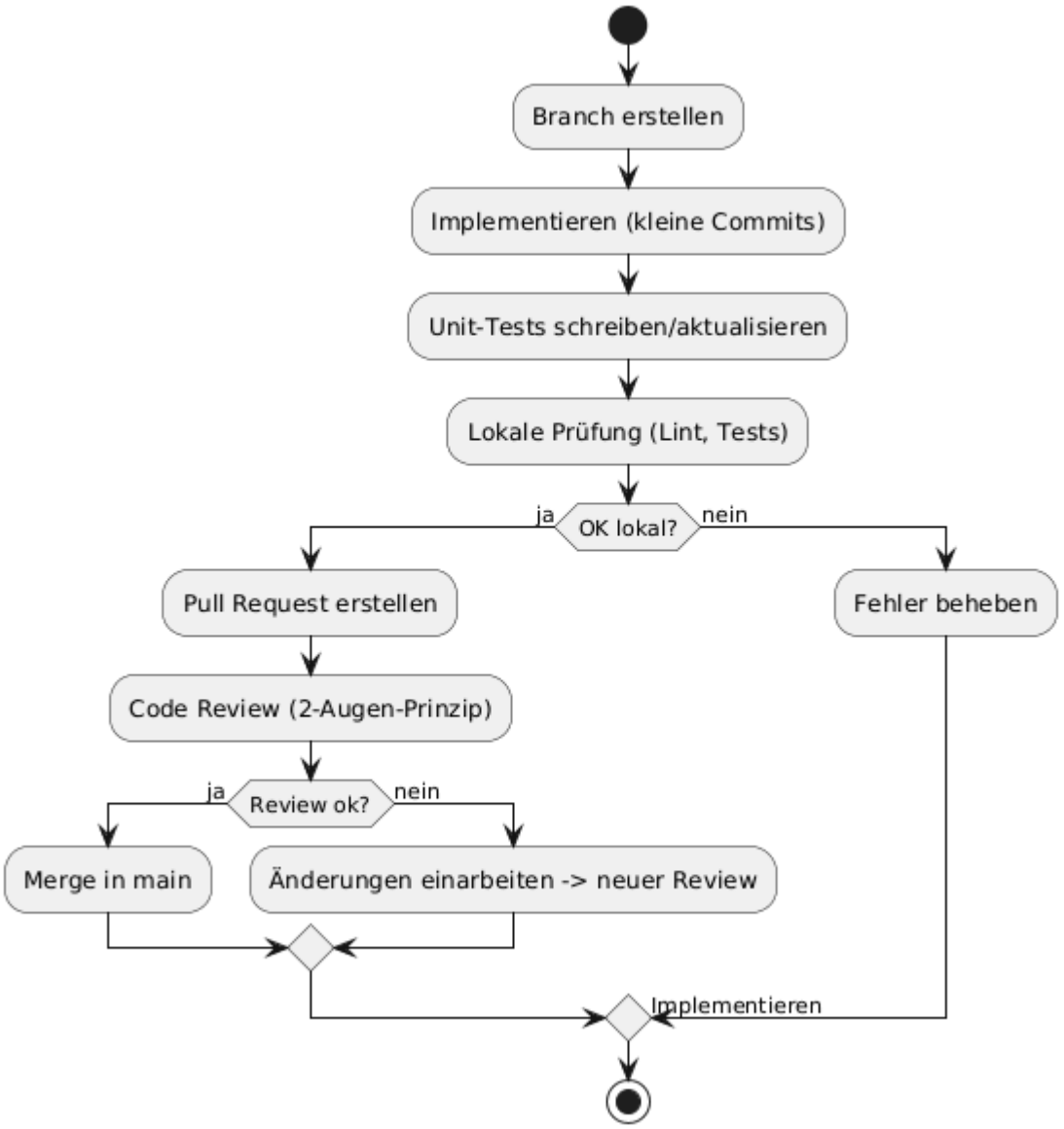
3) Sprint Planning



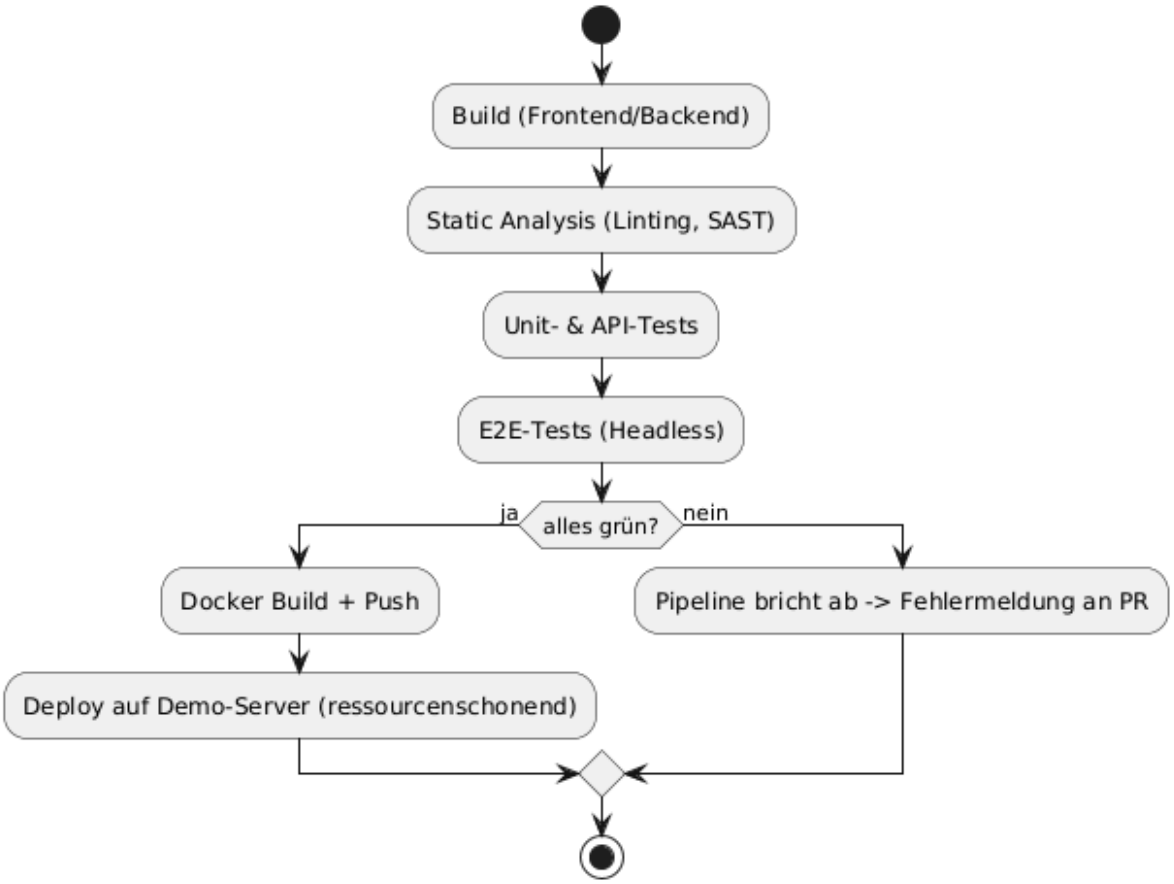
4) Daily Scrum



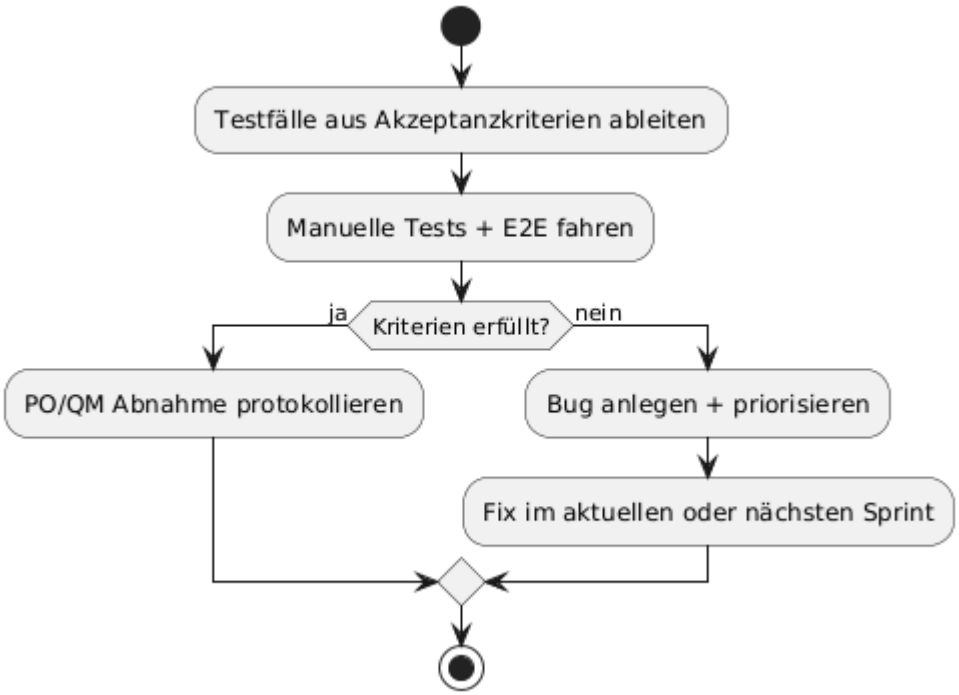
5) Implementierung (Dev-Flow je Task)



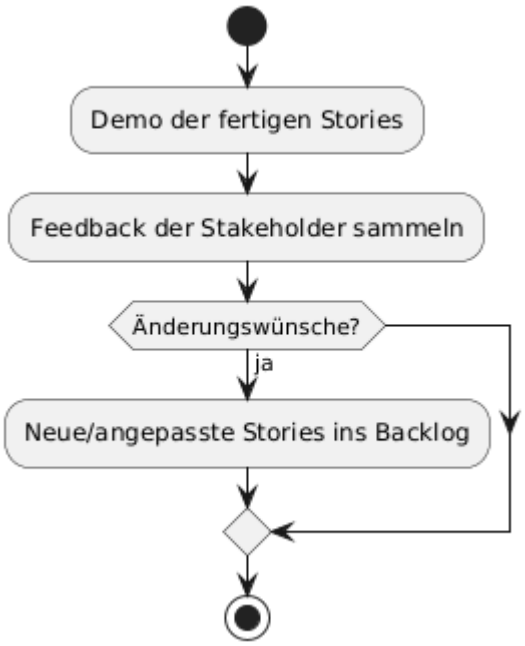
6) CI/CD-Pipeline



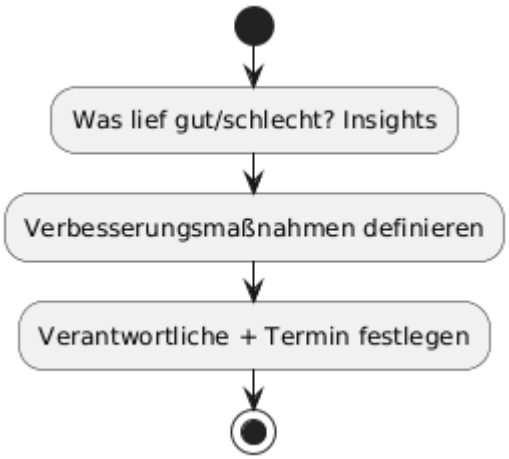
7) Test & Abnahme je Story



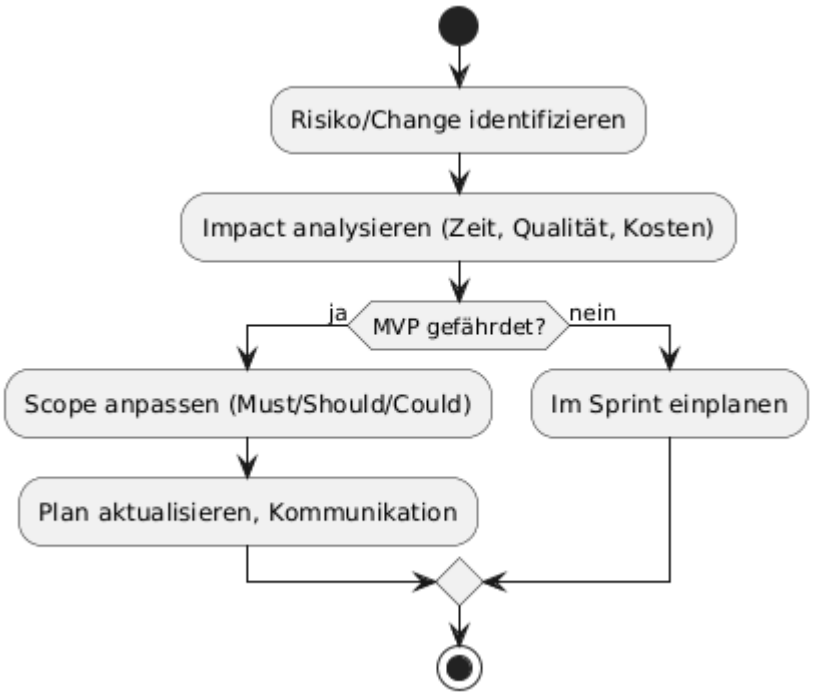
8) Sprint Review



9) Sprint Retrospektive



10) Change-/Risikomanagement (parallel)



Changelog:

10.10.2025:

Erstellung der Datei und grobe Vorstellungen